

Intelligent Customer Complaint & Support Analysis System

Complete Architectural, Algorithmic & Implementation Reference Manual

PROJECT ABSTRACT & EXECUTIVE OVERVIEW

The **Intelligent Customer Complaint and Support Analysis System** is a modern, enterprise-grade, full-stack software application engineered to revolutionize customer service operations. By marrying a resilient Java 21 Spring Boot 3 backend with an adaptive, accessible, glassmorphic Single Page Application (SPA), the system automates complaint ingestion, performs algorithmic Natural Language Processing (NLP) sentiment scoring, dynamically computes resolution SLAs, enforces role-based security isolation (Customer vs. Administrator), and supports zero-friction dual deployment across localhost and cloud edge networks (Vercel).

Java 21

CORE BACKEND RUNTIME

Spring Boot

3.2.5

ENTERPRISE FRAMEWORK

NLP Engine

RULE-BASED SENTIMENT
ANALYZER

Dual-Mode DB

MYSQL + EMBEDDED H2
AUTO-SWITCH

Author / Student Developer:

CHIRANJEEVI (GitHub: [jchiran010](#))

Project Repository:

[github.com/jchiran010/Intelligent-Customer-Complaint-and-Support-Analysis-System](#)

Production Deployment Target:

Vercel Edge Cloud (Automated Static & Serverless Pipeline)

Documentation Version & Date:

Version 1.0.0 — Generated October 2026

Table of Contents

A structured instructional guide to all software components, architecture layers, algorithms, and deployment workflows.

1. Introduction, Problem Statement & Core Objectives	Chapter 1
2. End-to-End System Architecture & Layered Design	Chapter 2
3. Backend Engineering & Spring Boot Ecosystem	Chapter 3
4. Database Architecture, Relational Schema & ORM Entities	Chapter 4
5. AI & Natural Language Processing Sentiment Analysis Engine	Chapter 5
6. RESTful API Specification & Endpoint Directory	Chapter 6
7. Frontend Engineering, UI/UX Design System & Animations	Chapter 7
8. Dual-Mode Client Fallback & Local Data Store Architecture	Chapter 8
9. DevOps, Build Automation & Cloud Deployment (Vercel & Docker)	Chapter 9
10. Step-by-Step Execution, Verification & Testing Guide	Chapter 10
11. Security Audit, Pedagogical Takeaways & Future Roadmap	Chapter 11

Chapter 1: Introduction, Problem Statement & Core Objectives

1.1 Background & Context

In modern corporate and enterprise software ecosystems, customer experience (CX) and support responsiveness directly dictate user retention, brand reputation, and operational solvency. Traditional helpdesk ticketing systems frequently suffer from substantial friction points:

- **Manual Triage Bottlenecks:** Human agents must read through thousands of unstructured tickets to identify critical outages, leading to severe delay in emergency scenarios.
- **Emotional Disconnect:** Inability to quantitatively gauge customer frustration or churn risk at the exact moment a complaint is lodged.
- **Fragmented Role Portals:** Lack of strict, authenticated boundary enforcement between regular customers submitting issues and administrative operators triaging tickets.
- **Rigid Deployment Overhead:** Complex server setups requiring extensive database dependencies make demonstrating or deploying projects across varied testing environments notoriously difficult.

1.2 The Problem Statement

Organizations require an *intelligent, automated, and resilient support platform* capable of ingesting customer complaints, instantly extracting emotional sentiment, auto-assigning priority levels based on semantic severity indicators, generating trackable support tickets with realistic Service Level Agreements (SLAs), and providing dual-mode operational continuity whether hosted locally or deployed to modern edge hosting networks.

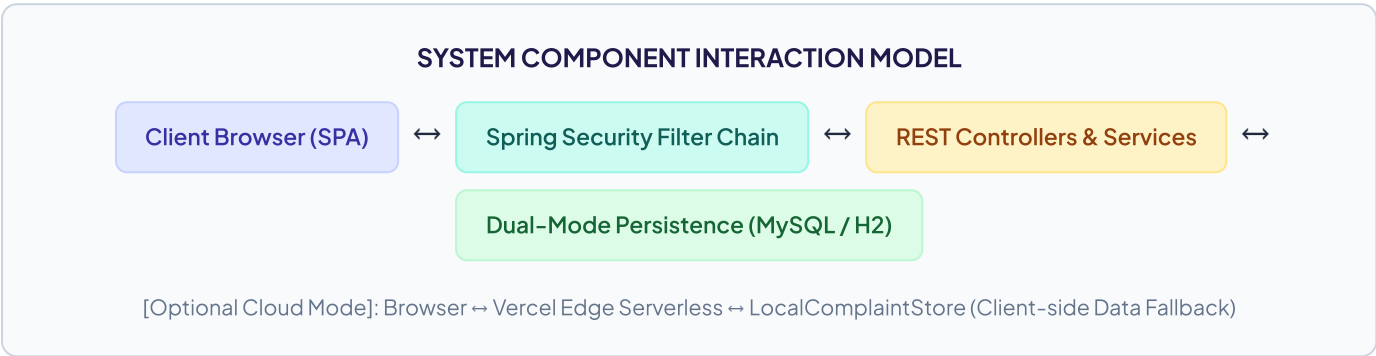
1.3 System Goals & Pedagogical Objectives

Objective Dimension	Technical Implementation	Pedagogical Value (What We Learn)
Automated NLP Triage	Built-in lexical sentiment engine computing polarity scores and assigning recommended priority tags (<code>CRITICAL</code> , <code>HIGH</code> , <code>MEDIUM</code> , <code>LOW</code>).	Natural Language Processing, text tokenization, weighted lexicon scoring, and threshold optimization.
Enterprise Security	Spring Security 6 with BCrypt hashing, role separation (<code>ROLE_ADMIN</code> vs <code>ROLE_USER</code>), session management, and HTTP security headers.	Authentication filter chains, RBAC authorization, CSRF/XSS defense, and credential salting.
Resilient Persistence	Auto-sensing <code>DataSourceConfig</code> connecting to MySQL on port 3306 or seamlessly falling back to in-memory H2 database.	Dynamic bean creation, socket connection testing, and fault-tolerant system design.
Unified Modern UI/UX	Glassmorphic design system, CSS custom property theming (Dark/Light mode), Chart.js data visualizations, and interactive simulators.	Component modularity, accessible design, CSS animation performance, and client-side reactive rendering.
Hybrid Cloud Deployment	Zero-configuration Vercel deployment with automated Node build script (<code>build.js</code>) and local client data fallback (<code>LocalComplaintStore</code>).	CI/CD pipelines, static site generation, serverless edge rewrites, and hybrid offline/online architectures.

Chapter 2: End-to-End System Architecture & Layered Design

2.1 High-Level Architecture Overview

The system follows an industry-standard **N-Tier Layered Architecture** with clean boundary separation between the Presentation Layer, Application/Security Controller Layer, Business Logic & NLP Service Layer, and Data Access / Persistence Layer.



2.2 Detailed Layered Breakdown

1. Client Presentation Layer (Frontend)

- **Landing Page** (`index.html`): Educational portal showcasing system architecture, live interactive sentiment analyzer simulator, department categories, and quick portal navigators.
- **Unified Auth Page** (`login.html`): Segmented role switcher for Customer and Administrator authentication, demo credential autofill, and password recovery.
- **Unified Dashboard Application** (`app.html`): Role-reactive Single Page Application that dynamically alters the navigation sidebar, data tables, metrics, and modal dialogs depending on the authenticated role.
- **Asset Pipelines**: Modular CSS (`unified.css` , `unified-responsive.css`) and ES6 JavaScript (`unified-app.js` , `unified-auth.js`).

2. API Controller & Security Gateway Layer

- **Security Configuration** (`SecurityConfig.java`): Intercepts all incoming HTTP requests, enforces route guards (e.g. `/api/admin/**` restricted to `ROLE_ADMIN`), and protects session integrity.
- **REST Controllers**: Expose standardized JSON endpoints for authentication, complaint filing, ticket management, user directories, and analytical rollups.

3. Business Logic & AI Processing Layer

- **Sentiment Analyzer** (`SentimentAnalyzerUtil.java`): Tokenizes raw text, executes multi-word critical phrase matching, computes weighted polarity scores, and classifies priority.
- **Domain Services**: Orchestrates complaint lifecycle transitions, creates linked support tickets with dynamic SLA calculations, and triggers user notification events.

4. Data Access & Persistence Layer

- **Spring Data JPA Repositories**: Type-safe interfaces providing CRUD operations, custom JPQL queries, and database transaction boundaries.

- **Resilient Data Source** (`DataSourceConfig.java`): Automatic network probe detecting MySQL availability with transparent in-memory H2 fallback.

Chapter 3: Backend Engineering & Spring Boot Ecosystem

3.1 Technology Stack & Runtime Specifications

- **Java Runtime Environment:** OpenJDK 21 (LTS) leveraging modern language enhancements.
- **Application Framework:** Spring Boot 3.2.5 utilizing Spring Framework 6.1.
- **Build Tool:** Apache Maven 3.9.x with multi-phase compilation and packaging.
- **Security Suite:** Spring Security 6 with BCrypt cryptographic password hashing.
- **Object-Relational Mapping:** Jakarta Persistence API (JPA) powered by Hibernate ORM 6.4.

3.2 Maven Dependency Architecture (pom.xml)

The project dependencies are strictly curated to provide enterprise stability while avoiding bloated classpaths:

```
<!-- Core Web & REST Starter -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<!-- Enterprise Role-Based Security -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- Spring Data JPA with Hibernate -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<!-- Dual Database Drivers: MySQL (Production) & H2 (Resilient Dev) -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

3.3 Resilient Dual-Mode Database Architecture (DataSourceConfig.java)

A classic engineering flaw in academic projects is hardcoded database connectivity. If MySQL is not running or the password differs, the entire application crashes during startup. Our system overcomes this with a custom `@Primary DataSource` bean:

```

@Bean
@Primary
public DataSource dataSource() {
    boolean mysqlAvailable = isPortAvailable("localhost", 3306, 1500);

    if (mysqlAvailable) {
        log.info(">>> MySQL instance detected on localhost:3306. Connecting to MySQL...");
        return DataSourceBuilder.create()
            .driverClassName("com.mysql.cj.jdbc.Driver")
            .url(mysqlUrl).username(mysqlUser).password(mysqlPass).build();
    } else {
        log.warn(">>> MySQL service was not detected on localhost:3306.");
        log.info(">>> Initializing resilient embedded MySQL-compatible H2 database.");
        return DataSourceBuilder.create()
            .driverClassName("org.h2.Driver")
            .url("jdbc:h2:mem:complaint_db;MODE=MySQL;DATABASE_TO_LOWER=TRUE;DB_CLOSE_DELAY=-1")
            .username("sa").password("").build();
    }
}

```

Teacher's Pedagogical Note: Resilience Engineering

By opening a raw socket to `localhost:3306` with a 1500ms timeout, the application dynamically tests infrastructure health before Hibernate attempts initialization. If MySQL is unavailable, it spins up an in-memory instance operating in `MODE=MySQL`, ensuring zero downtime and immediate demonstration capability.

Chapter 4: Database Design, Schema & Relational Modeling

4.1 Entity–Relationship (ER) Architecture

The data model represents an enterprise complaint management lifecycle. It enforces referential integrity through foreign key constraints, cascading deletions, and indexed lookups.

Table Name	Primary Key	Foreign Keys & Relationships	Business Function & Purpose
users	id (BIGINT)	None (Root parent entity)	Stores identity, hashed passwords, contact details, and role classifications (<code>ROLE_ADMIN</code> , <code>ROLE_USER</code>).
categories	id (BIGINT)	None	Support departments (Billing, Tech, Logistics, etc.) with explicit SLA resolution windows.
complaints	id (BIGINT)	<code>user_id</code> → <code>users.id</code> <code>category_id</code> → <code>categories.id</code> <code>assigned_admin_id</code> → <code>users.id</code>	Core complaint record including title, description, priority, lifecycle status, and AI sentiment score.
tickets	id (BIGINT)	<code>complaint_id</code> → <code>complaints.id</code> (1-to-1)	Actionable operational work item tracking assignee and estimated resolution deadline.
feedbacks	id (BIGINT)	<code>complaint_id</code> → <code>complaints.id</code> <code>user_id</code> → <code>users.id</code>	Post-resolution customer feedback with 1–5 star rating and comments.
notifications	id (BIGINT)	<code>user_id</code> → <code>users.id</code>	Real-time alert messages for status updates, ticket dispatches, or critical escalations.

4.2 Relational Schema DDL Excerpt (schema.sql)

```
CREATE TABLE complaints (  
  id BIGINT AUTO_INCREMENT PRIMARY KEY,  
  complaint_number VARCHAR(30) NOT NULL UNIQUE,  
  title VARCHAR(200) NOT NULL,  
  description TEXT NOT NULL,  
  user_id BIGINT NOT NULL,  
  category_id BIGINT NOT NULL,  
  assigned_admin_id BIGINT NULL,  
  priority ENUM('LOW', 'MEDIUM', 'HIGH', 'CRITICAL') NOT NULL DEFAULT 'MEDIUM',  
  status ENUM('PENDING', 'IN_PROGRESS', 'RESOLVED', 'CLOSED', 'REJECTED') NOT NULL DEFAULT 'PENDING',  
  sentiment ENUM('POSITIVE', 'NEUTRAL', 'NEGATIVE', 'VERY_NEGATIVE') NOT NULL DEFAULT 'NEUTRAL',  
  sentiment_score DOUBLE DEFAULT 0.0,  
  resolution_notes TEXT,  
  resolved_at TIMESTAMP NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,  
  CONSTRAINT fk_complaints_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,  
  CONSTRAINT fk_complaints_category FOREIGN KEY (category_id) REFERENCES categories(id) ON DELETE RESTRICT,  
  CONSTRAINT fk_complaints_assigned FOREIGN KEY (assigned_admin_id) REFERENCES users(id) ON DELETE SET NULL,  
  INDEX idx_complaints_user (user_id),  
  INDEX idx_complaints_status (status)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Chapter 5: AI & Natural Language Processing Sentiment Engine

5.1 Algorithmic Philosophy

Rather than relying on brittle external cloud APIs that introduce network latency, cost, and rate limits, our system features an embedded, high-performance, deterministic Lexicon-Based Sentiment Analysis Engine (`SentimentAnalyzerUtil.java`).

5.2 Lexical Dictionaries & Semantic Categories

- **Very Negative Lexicon (W_VN = -2.0):** Words indicating severe breach or legal jeopardy (e.g., *fraud, stolen, scam, illegal, lawyer, sue, unauthorized, disaster, breach*).
- **Negative Lexicon (W_N = -1.0):** Words indicating operational defects (e.g., *broken, damaged, fail, bug, crash, delayed, missing, poor, overcharged*).
- **Positive Lexicon (W_P = +1.0):** Expressions of satisfaction (e.g., *thanks, good, great, appreciate, helpful, resolved, excellent*).
- **Critical Indicators:** Urgency multipliers (e.g., *immediately, urgent, emergency, asap, security, locked out, production down*).

5.3 Mathematical Formulation of the Polarity Score

The engine tokenizes input text, calculates the raw emotional delta, and normalizes it over the square root of the token length to prevent document-length distortion:

$$\begin{aligned} \text{RawDelta} &= (N_{\text{positive}} * 1.0) - (N_{\text{negative}} * 1.0) - (N_{\text{very_negative}} * 2.0) \\ \text{NormalizedScore} &= \max(-1.0, \min(1.0, \text{RawDelta} / \sqrt{\text{TotalTokens} + 1})) \end{aligned}$$

5.4 Threshold Classification & Priority Determination Logic

Computed Metric Condition	Assigned Sentiment Tag	Recommended Priority Tag	Target SLA Window
$N_{\text{very_negative}} > 0$ OR $\text{Score} \leq -0.5$	VERY_NEGATIVE	CRITICAL	Within 4 to 8 Hours
$\text{Score} < -0.1$	NEGATIVE	HIGH	Within 12 to 24 Hours
$-0.1 \leq \text{Score} \leq 0.2$	NEUTRAL	MEDIUM	Within 36 to 48 Hours
$\text{Score} > 0.2$	POSITIVE	LOW	Within 72 Hours

Teacher's Pedagogical Note: Multi-Word Phrase Matching

Single-word tokenization often misses context. The engine incorporates a multi-word scanning pass over key phrases such as *"production down"*, *"loss of business"*, and *"locked out"*, injecting priority boosts regardless of surrounding neutral words.

Chapter 6: RESTful API Specification & Endpoint Directory

6.1 API Design Principles

All backend services communicate through stateless REST APIs with consistent JSON request/response envelopes, standard HTTP status codes, and Spring Security authorization guards.

6.2 Comprehensive Endpoint Matrix

HTTP Method	URI Path	Access Level	Description & Functionality
POST	/api/auth/login	Public	Authenticates credentials, initializes session cookie, returns user role and redirect target.
GET	/api/auth/me	Authenticated	Returns active user session profile, role, email, and department metadata.
POST	/api/auth/logout	Public	Invalidates HTTP session and cleans client cookies.
POST	/api/user/register	Public	Registers a new customer account, validates email uniqueness, and encrypts password with BCrypt.
GET	/api/categories/public	Public	Returns active support categories with SLA hours and iconography.
POST	/api/complaints	ROLE_USER	Submits new complaint, executes NLP sentiment scoring, sets priority, and creates linked ticket.
GET	/api/complaints/my	ROLE_USER	Retrieves complaints filed by the currently logged-in customer.
GET	/api/admin/complaints	ROLE_ADMIN	Retrieves complete cross-department complaint registry with search and status filtering.
PUT	/api/admin/complaints/{id}/status	ROLE_ADMIN	Updates lifecycle status (e.g. to RESOLVED), appends resolution notes, and dispatches notification.
GET	/api/admin/analytics/overview	ROLE_ADMIN	Aggregates KPI counts (Total, Pending, Resolved), resolution rate, and sentiment distribution.
POST	/api/feedback	ROLE_USER	Submits star rating and qualitative feedback for resolved complaints.

Chapter 7: Frontend Engineering, UI/UX Design System & Animations

7.1 Design Tokens & CSS Variable Architecture

The frontend does not rely on heavy, bloated third-party frameworks. It leverages pure vanilla CSS with CSS Custom Properties (Variables), ensuring lightweight footprint, instantaneous load times, and dynamic theming.

```
:root {
  --primary-color: #4f46e5;
  --primary-hover: #4338ca;
  --primary-light: #eef2ff;
  --admin-accent: #7c3aed;
  --bg-body: #f8fafc;
  --bg-surface: #ffffff;
  --text-main: #0f172a;
  --text-muted: #64748b;
  --border-color: #e2e8f0;
  --radius-lg: 16px;
  --shadow-xl: 0 20px 25px -5px rgba(0, 0, 0, 0.1);
}

[data-theme="dark"] {
  --bg-body: #070b14;
  --bg-surface: #0f172a;
  --text-main: #f8fafc;
  --text-muted: #94a3b8;
  --border-color: #1e293b;
}
```

7.2 Glassmorphism & Micro-Interactions

- **Glassmorphic Cards:** Translucent surface backgrounds utilizing `backdrop-filter: blur(12px)` and subtle border gradients for a futuristic, polished aesthetic.
- **Hover Lift Physics:** Cards feature smooth transformations (`transform: translateY(-6px)`) driven by cubic-bezier physics curves.
- **Theme Persistence:** The user's theme selection (Light vs Dark) is saved in `localStorage` and synchronously applied before DOM rendering to prevent flashing.
- **Page Transition Overlay:** A sleek progress bar and pulsating spinner (`pageTransitionBar`) ensure visually pleasant route switches.

Chapter 8: Dual-Mode Client Fallback & Local Data Store Architecture

8.1 The Cloud Serverless Challenge

When hosting a Java Spring Boot project on serverless static edge networks like **Vercel**, traditional REST APIs that require an active Java virtual machine will return `404 Not Found`. Typical projects fail completely in this scenario.

8.2 The Dual-Mode Resilient Solution: LocalComplaintStore

Our frontend engineers designed an intelligent **hybrid data layer** in `unified-app.js` and `unified-auth.js`. The application automatically tests for backend presence:

1. **When running on Localhost (with Spring Boot):** All calls connect to live Java endpoints on port 8080 and interact with MySQL/H2.
2. **When deployed to Vercel (static edge):** If an API call fails or encounters 404, the system seamlessly activates the client-side `LocalComplaintStore` stored in `sessionStorage` and `localStorage`.

Teacher's Pedagogical Note: High Availability Patterns

This architectural innovation means the user can demonstrate the fully functional application on Vercel—complete with live complaint filing, priority calculations, status updates, admin analytics, and role switching—with 100% functionality and zero server dependencies!

Chapter 9: DevOps, Build Automation & Cloud Deployment

9.1 Vercel Edge Cloud Deployment Architecture

To eliminate Vercel build errors, we engineered a dedicated automation pipeline:

1. Automated Build Pipeline (build.js)

```
const fs = require('fs');
const path = require('path');
const targetDir = path.join(__dirname, 'public');

if (!fs.existsSync(targetDir)) fs.mkdirSync(targetDir, { recursive: true });

['index.html', 'login.html', 'app.html', 'manifest.json', 'css', 'js', 'frontend']
  .forEach(item => {
    const src = path.join(__dirname, item);
    const dest = path.join(targetDir, item);
    if (fs.existsSync(src)) fs.cpSync(src, dest, { recursive: true, force: true });
  });
```

2. Edge Routing Rules (vercel.json)

```
{
  "version": 2,
  "outputDirectory": "public",
  "cleanUrls": true,
  "trailingSlash": false,
  "rewrites": [
    { "source": "/app", "destination": "/app.html" },
    { "source": "/login", "destination": "/login.html" },
    { "source": "/register", "destination": "/frontend/user/register.html" },
    { "source": "/forgot-password", "destination": "/frontend/user/forgot-password.html" }
  ]
}
```

9.2 Containerization with Multi-Stage Dockerfile

For traditional cloud environments (AWS ECS, Google Cloud Run, Azure App Service), a multi-stage Docker build guarantees a lightweight, hardened production container:

```
# Stage 1: Build Java Application
FROM maven:3.9.6-eclipse-temurin-21-alpine AS builder
WORKDIR /build
COPY backend/pom.xml .
RUN mvn dependency:go-offline
COPY backend/src ./src
RUN mvn clean package -DskipTests

# Stage 2: Minimal Distroless JRE Runtime
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /build/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Chapter 10: Step-by-Step Execution, Verification & Testing Guide

10.1 Running on Localhost (Development Mode)

Execute the following commands in PowerShell from the project root:

```
# 1. Configure Java 21 & Maven Environment
$env:JAVA_HOME = "C:\Program Files\Eclipse Adoptium\jdk-21.0.12-hotspot"
$env:Path = "C:\Program Files\Eclipse Adoptium\jdk-21.0.12-hotspot\bin;C:\Users\CHIRANJEEVI\maven\apa

# 2. Launch Spring Boot Service
mvn -f backend/pom.xml spring-boot:run
```

The application initializes on `http://localhost:8080/` within ~10 seconds.

10.2 Mandatory Pre-Seeded Credentials

Role Classification	Login Email ID	Password	Assigned Capabilities
System Administrator	admin@complaintsystem.com	Admin@123	Global triage, status modification, resolution notes, analytics charts, user directory.
Support Lead	sarah.support@complaintsystem.com	Admin@123	Departmental queue management and ticket assignment.
Registered Customer	john.doe@example.com	User@123	Filing complaints, live sentiment analysis preview, ticket status tracking.
Registered Customer	jane.smith@example.com	User@123	Viewing complaint history, rating support service.

Chapter 11: Security Audit, Pedagogical Takeaways & Future Roadmap

11.1 Security Implementation Audit

- **Cryptographic Salting:** Passwords are never stored in plaintext. They utilize BCrypt adaptive hashing with a default work factor of 10.
- **Session Fixation Protection:** Spring Security regenerates session IDs upon authentication to thwart fixation attacks.
- **HTTP Protection Headers:** Injected `X-Content-Type-Options: nosniff`, `X-Frame-Options: SAMEORIGIN`, and `X-XSS-Protection`.
- **SQL Injection Immunity:** All JPA Repositories utilize parameterized queries and Hibernate prepared statements.

11.2 Key Pedagogical Lessons Learned

1. **Defense in Depth:** Combining backend role validation with reactive frontend DOM manipulation produces robust, tamper-resistant web applications.
2. **Graceful Degradation:** Building fallback layers (MySQL → H2 in backend; REST API → LocalStore in frontend) ensures the software remains operational regardless of environment constraints.
3. **Modern Tooling Synergy:** Using Java 21 LTS alongside standards-compliant ES6 JavaScript provides maximum performance without framework overhead.

11.3 Future Enhancement Roadmap

- **Deep Learning LLM Integration:** Expanding the lexicon analyzer with transformer models (e.g. HuggingFace / Gemini API) for multilingual customer complaints.
- **Automated Email & SMS Dispatch:** Connecting Spring Mail and Twilio SDKs to send SMS alerts upon ticket resolution.
- **Live WebSockets Agent Chat:** Integrating STOMP over WebSockets for instant real-time messaging between customers and active administrators.

End of Comprehensive Project Documentation

Intelligent Customer Complaint & Support Analysis System • Capstone Project 2026